1 Write a Python program to implement a class BankAccount with deposit

and withdraw methods.

Aim

To write a Python program that defines a BankAccount class with methods to deposit and withdraw money, demonstrating object-oriented programming concepts.

⚙ Algorithm

1. Define a class .
2. Initialize attributes: and .
3. Create a method to add money to the balance.
4. Create a method to deduct money if sufficient balance exists.
5. Create a method to show the current balance.
6. Create objects and demonstrate deposit and withdrawal operations.

CODE

```
# Program to implement a BankAccount class


class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance


    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Deposit amount must be positive.")
```

```
    def withdraw(self, amount):
        if amount > 0:
            if self.balance >= amount:
                self.balance -= amount
                print(f"Withdrew: {amount}")
            else:
                print("Insufficient balance!")
        else:
            print("Withdrawal amount must be positive.")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}, Balance: {self.balance}")



# Demonstration
account1 = BankAccount("Yogin", 1000)

account1.display_balance()
account1.deposit(500)
account1.withdraw(300)
account1.withdraw(1500)  # Attempting more than balance
account1.display_balance()
```

OUTPUT

```
Account Holder: Yogin, Balance: 1000
Deposited: 500
Withdrew: 300
Insufficient balance!
Account Holder: Yogin, Balance: 1200
```

Conclusion

- The program defines a BankAccount class with deposit and withdraw methods.
- It ensures validation (positive amounts, sufficient balance).
- Demonstrates object-oriented programming concepts like encapsulation and methods.

2 Write a program to calculate compound interest using functions.

Aim

To write a Python program that calculates compound interest using a user-defined function.

⚙ Algorithm

1. Define a function where:
- = initial amount (P)
- = annual interest rate (R in %)
- = number of years (T)
- = number of times interest is compounded per year
2. Convert rate into decimal form ().
3. Apply the formula:

4. Return the compound interest.
5. Call the function with sample values and display the result.

CODE

```
# Function to calculate compound interest
def compound_interest(principal, rate, time, n):
```

```
    # Convert rate to decimal
    rate = rate / 100
    # Compound interest formula
    amount = principal * (1 + rate/n) ** (n*time)
    ci = amount - principal
    return ci, amount

# Sample input
P = float(input("Enter Principal Amount: "))
R = float(input("Enter Annual Interest Rate (%): "))
T = int(input("Enter Time (in years): "))
N = int(input("Enter number of times interest is compounded per year: "))

# Function call
ci, total = compound_interest(P, R, T, N)

print(f"\nCompound Interest: {ci:.2f}")
print(f"Total Amount after {T} years: {total:.2f}")
```

OUTPUT

```
Enter Principal Amount: 1000
Enter Annual Interest Rate (%): 5
Enter Time (in years): 2
Enter number of times interest is compounded per year: 4

Compound Interest: 104.49
Total Amount after 2 years: 1104.49
```

Conclusion

- The program successfully calculates compound interest using a function.
- It applies the standard formula with user inputs for flexibility.
- This demonstrates how functions make code reusable and organized.